

AI in Robotics (MCTR912)

Milestone 3

Team 5

Bassam Walid 55-0499

Styven Hany 55-10067

Youssef Mohamed 55-1031

Verina Maged 55-5282

a) Content from Milestone 2 (Summary)

In Milestone 2, a symbolic planning framework was developed for an indoor service robot operating in a domestic apartment environment. A knowledge base (KB) was designed to represent rooms, objects, robot state, and connectivity constraints. A Planning Domain Definition Language (PDDL) domain and problem generator were implemented to allow the robot to reason about navigation and manipulation tasks. The Fast Downward planner was used to compute valid plans based on user-defined goals entered through a command-line interface (CLI). The planner output was parsed and sequentially executed, updating the knowledge base to reflect the new world state after each action.

b) ROS2 Architecture Description

The system architecture follows a modular ROS2-based design, integrating symbolic planning with robot navigation and simulation.

Nodes

- CLI Node (Python): Accepts user goals (e.g., object relocation or robot navigation) and triggers the planning process.
- Planner Node (Fast Downward): Generates a symbolic plan from the PDDL domain and problem files.
- Plan Executor Node: Interprets planner actions and translates navigation actions into Nav2 commands while updating the knowledge base.
- Nav2 Stack Nodes:
 - bt_navigator
 - planner_server
 - controller_server
 - map_server
 - amcl
- Gazebo Simulation Node: Simulates the apartment environment, robot, and objects.

Topics and Actions

- /navigate_to_pose (Nav2 action): Used by the executor to send navigation goals.
- /map, /odom, /tf: Standard localization and mapping topics used by Nav2.
- /cmd_vel: Velocity commands generated internally by the Nav2 controller.
- /gazebo/model_states: Used for monitoring simulated entities.

Launch Files

- Gazebo Launch: Launches the TurtleBot3 with OpenManipulator in a house environment.
- Navigation Launch: Starts the Nav2 stack with a pre-generated map.
- Spawn Script: Spawns static object models in Gazebo at predefined coordinates.

This architecture ensures a clean separation between symbolic reasoning, navigation, execution, and simulation.

d) Planner and Algorithm Description

The system uses **Fast Downward**, a state-of-the-art classical planner, to compute plans. The planning domain is defined in PDDL and includes the following action types:

- move: Navigate between connected rooms.
- pick: Pick up an object in the current room.
- place: Place a held object in a target room.

The planner operates under deterministic assumptions and generates sequential plans using heuristic forward search. The problem file is generated dynamically based on the current knowledge base state and user-defined goals. The resulting plan is parsed and executed step-by-step by the executor node.

c) World Modelling

The world model used in this project represents a domestic apartment environment composed of four rooms and a set of household objects. The symbolic representation of rooms, connectivity, and robot state is consistent with the model developed in Milestone 2. The primary extension introduced in this milestone is the addition of explicit physical coordinates for objects to align symbolic reasoning with the simulated environment.

Rooms and Room Coordinates

The apartment consists of the following rooms:

- Kitchen
- Living Room
- Bedroom
- Bathroom

The reference coordinates of each room are identical to those defined in Milestone 2 and are used as navigation goals by the robot through the Nav2 stack.

Room	Coordinates (x, y)
Kitchen	(-4.5, 0.0)
Living Room	(-0.5, 4.0)
Bedroom	(7.0, 1.5)
Bathroom	(8.0, -1.0)

Room Connectivity

The connectivity between rooms remains unchanged from Milestone 2 and is defined as a bidirectional graph:

- Kitchen $\leftrightarrow$ Living Room
- Living Room $\leftrightarrow$ Bedroom
- Bedroom $\leftrightarrow$ Bathroom

These constraints restrict valid navigation actions during planning and execution.

Objects and Symbolic Locations

Eight household objects are included in the environment. Each object is associated symbolically with a room, representing its logical location within the apartment.

Object Physical Coordinates (New Addition)

To support simulation consistency and visualization in Gazebo, each object was assigned explicit physical coordinates (x, y, z) , where all objects are placed on the ground plane ($z = 0$). These coordinates were added to the knowledge base and used to spawn the objects in the Gazebo simulation.

This addition represents the main extension beyond Milestone 2, enabling alignment between the symbolic planner state and the physical simulation.

Object	Room	Coordinates (x, y, z)
cup	kitchen	(-4.0, 0.0, 0.0)
apple	kitchen	(-4.2, -2.0, 0.0)
bottle	livingroom	(-1.5, 4.2, 0.0)
remote	livingroom	(0.2, 4.5, 0.0)
laptop	livingroom	(1.7, 1.4, 0.0)
book	bedroom	(6.8, 1.3, 0.0)
shoe	bedroom	(3.2, 3.5, 0.0)
towel	bathroom	(8.0, -3.5, 0.0)

These object coordinates were directly derived from the knowledge base implementation and were used to spawn static object models in the Gazebo house environment. While physical grasping was abstracted at the symbolic level, the inclusion of object coordinates ensured consistency between planning, execution, and simulation.

e) System Performance Evaluation

The system successfully demonstrated end-to-end integration between symbolic planning, navigation, and simulated manipulation within the ROS 2 framework. Symbolic task goals were translated into PDDL problems and solved using a classical planner, producing valid action sequences. These planner actions were executed by an executor node, which correctly mapped symbolic navigation actions into Nav2 goal poses. As a result, the robot consistently navigated to the intended room locations within the simulated apartment environment, and Gazebo accurately reflected the robot's motion and pose updates.

All eight objects were successfully spawned in Gazebo at predefined locations corresponding to their symbolic room assignments in the knowledge base. This ensured consistency between the symbolic world model and the simulated environment. The executor correctly interpreted pick and place actions, updating the knowledge base to reflect object possession and placement states, thereby preserving logical correctness and plan consistency throughout execution.

To extend the system beyond abstract manipulation, motion control of the robot's manipulator was integrated using the MoveIt-compatible trajectory controllers provided with the TurtleBot3 Manipulation platform. Arm and gripper motions were successfully executed in Gazebo using joint trajectory and gripper command action interfaces. This enabled the system to perform physically observable pick and place motions, demonstrating full integration between symbolic planning, navigation, and low-level actuator control.

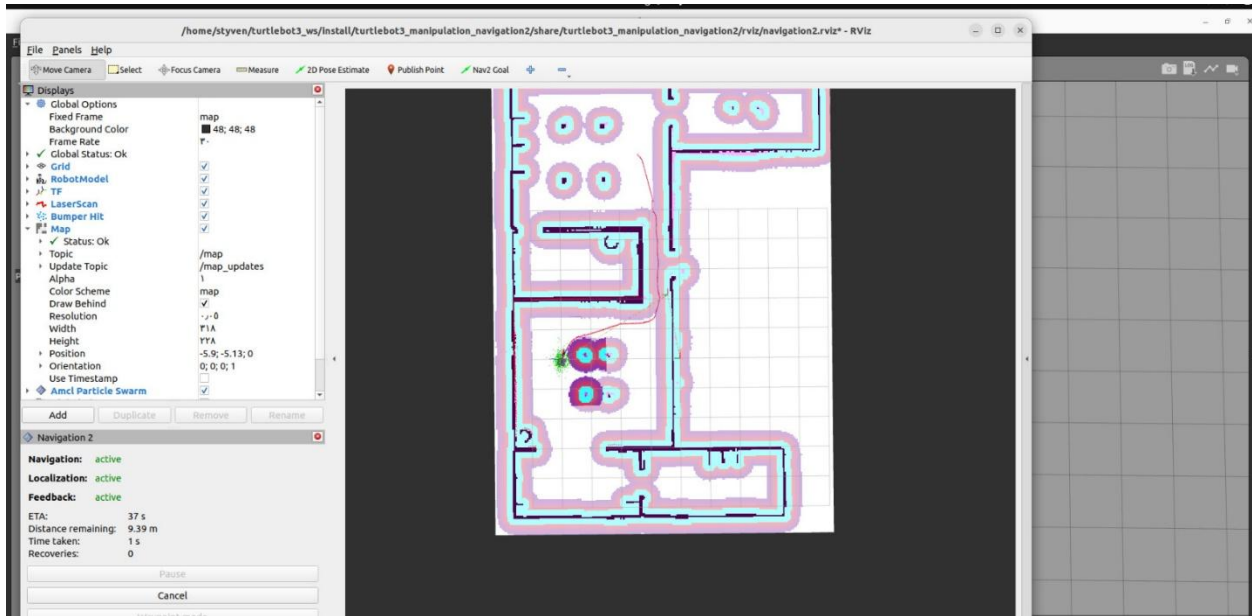
However, precise physical grasping of objects was not consistently achieved. Although object positions were defined symbolically and used to spawn models in Gazebo, discrepancies between Gazebo's ground-truth model poses, Nav2's localization estimates, and the robot's manipulator reference frame led to small spatial misalignments. As a result, the manipulator occasionally closed the gripper without making contact with the object (i.e., "picking air"). This limitation arises from the absence of a perception-driven object localization pipeline, as grasp execution relied on fixed, pre-tuned joint configurations rather than real-time object pose estimation in the robot's local frame.

Despite this limitation, the system successfully demonstrates the complete planning-execution pipeline, including symbolic reasoning, autonomous navigation, coordinated arm and gripper motion, and consistent knowledge base updates. The observed manipulation inaccuracies highlight a well-known challenge in robotics: bridging the gap between global navigation frames and local manipulation frames. Addressing this would require integrating

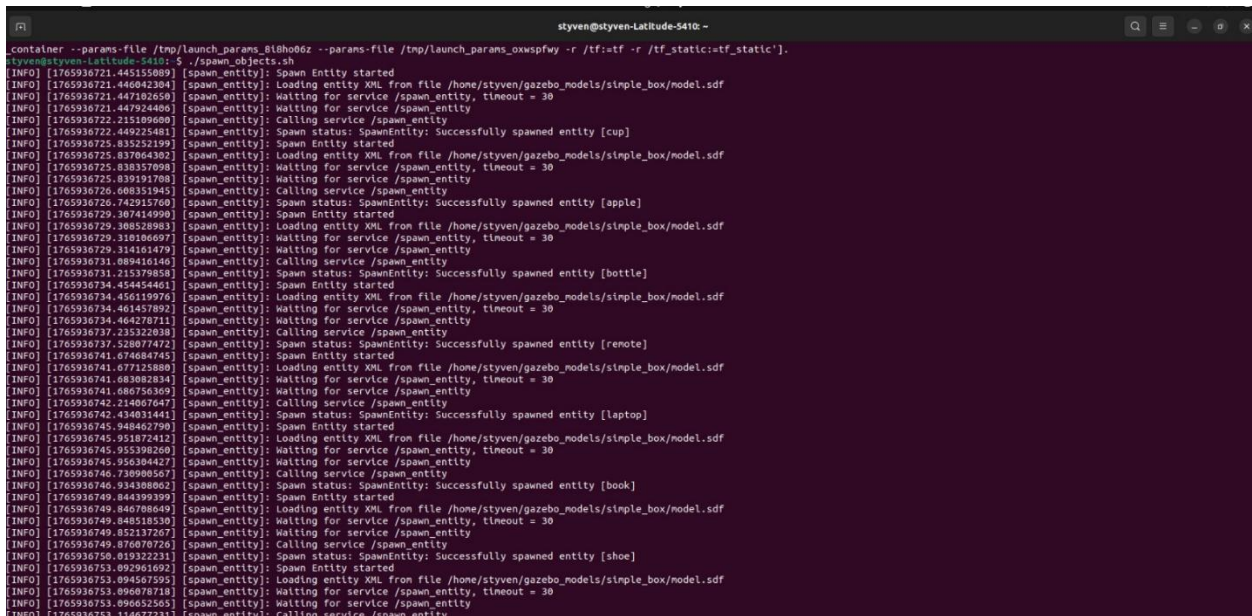
sensor-based perception (e.g., vision or fiducial markers) to estimate object poses relative to the robot prior to grasp execution, which is identified as future work.

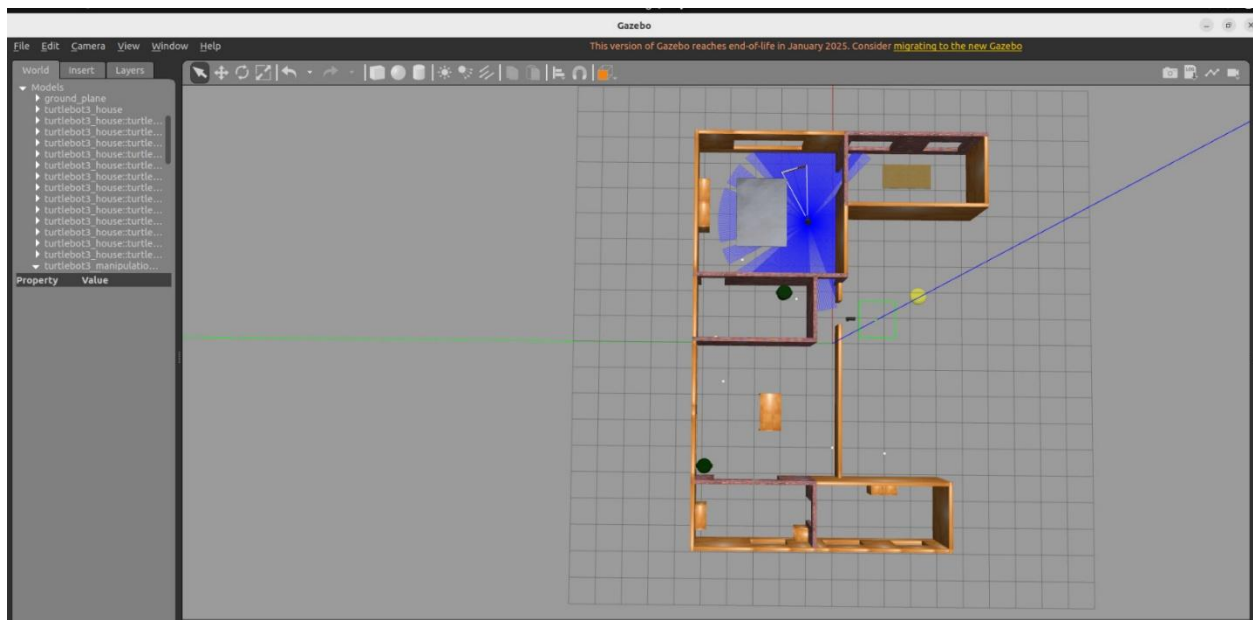
f) Simulation screenshots

Nav2 planning Motion:



Objects Spawning:





CLI (user input + planning + updating kb + sending coordinates to nav2):

```

styven@styven-Latitude-5410: ~/milestone 2/projects/downward/downward/src
[INFO] [1765936884.04832479] [plan_executor]: Waiting for Nav2...
[INFO] [1765936885.083947869] [basic_navigator]: ancl/get_state service not available, waiting...
[INFO] [1765936889.628853784] [basic_navigator]: Nav2 is ready for use!
[INFO] [1765936889.624060957] [plan_executor]: Nav2 is active.
Apartment Robot Planner - MCTS12 Milestone 2

=== CURRENT WORLD STATE ===
Robot: kitchen
Holding: None
Objects:
  cup: kitchen
  book: bedroom
  bottle: livingroom
  remote: livingroom
  towel: bathroom
  shoe: bedroom
  apple: kitchen
  laptop: livingroom
Connections:
  bathroom -> bedroom
  bedroom -> bathroom
  bedroom -> livingroom
  kitchen -> livingroom
  livingroom -> bedroom
  livingroom -> kitchen

=====
Enter goal (or 'quit'): at remote kitchen

PLAN (4 actions):
1. (move kitchen livingroom)
2. (pick remote livingroom)
3. (move livingroom kitchen)
4. (place remote kitchen)

[INFO] [1765936858.810050379] [plan_executor]: Navigating to livingroom (-0.5, 4)
[INFO] [1765936858.824360956] [basic_navigator]: Navigating to goal: -0.5 4.0...
[INFO] [1765936935.728318263] [plan_executor]: Arrived at livingroom
[INFO] [1765936935.728175377] [plan_executor]: Picking up remote (symbolic)
[INFO] [1765936935.728565422] [plan_executor]: Navigating to kitchen (-4.5, 0.0)
[INFO] [1765936935.733586670] [basic_navigator]: Navigating to goal: -4.5 0.0...
[INFO] [1765937019.898798535] [plan_executor]: Arrived at kitchen
[INFO] [1765937019.892372899] [plan_executor]: Placing remote in kitchen (symbolic)

=== CURRENT WORLD STATE ===
Robot: kitchen
Holding: None

```

```
styven@styven-Latitude-5410: ~/milestone 2/projects/downward/downward/jrc

1. (move livingroom bedroom)
2. (move bedroom bathroom)
3. (pick towel bathroom)
4. (move bathroom bedroom)
5. (move bedroom livingroom)
6. (place towel livingroom)
[INFO] [1765937446.756146005] [plan_executor]: Navigating to bedroom (7.0, 1.5)
[INFO] [1765937446.759728154] [basic_navigator]: Navigating to goal: 7.0 1.5...
[INFO] [1765937507.599882296] [plan_executor]: Arrived at bedroom
[INFO] [1765937507.601446844] [plan_executor]: Navigating to bathroom (8, -1.0)
[INFO] [1765937507.608353031] [basic_navigator]: Navigating to goal: 8.0 -1.0...
[INFO] [1765937649.419830262] [plan_executor]: Arrived at bathroom
[INFO] [1765937649.420964079] [plan_executor]: Picking up towel (symbolic)
[INFO] [1765937649.422504916] [plan_executor]: Navigating to bedroom (7.0, 1.5)
[INFO] [1765937649.424672212] [basic_navigator]: Navigating to goal: 7.0 1.5...
[INFO] [1765937704.616727548] [plan_executor]: Arrived at bedroom
[INFO] [1765937704.620966334] [plan_executor]: Navigating to livingroom (-0.5, 4)
[INFO] [1765937704.622919074] [basic_navigator]: Navigating to goal: -0.5 4.0...
[INFO] [1765937842.485780806] [plan_executor]: Arrived at livingroom
[INFO] [1765937842.487109037] [plan_executor]: Placing towel in livingroom (symbolic)

=== CURRENT WORLD STATE ===
Robot: livingroom
Holding: None
Objects:
  cup: kitchen
  book: bedroom
  bottle: livingroom
  remote: kitchen
  towel: livingroom
  shoe: bedroom
  apple: kitchen
  laptop: livingroom
Connections:
  bathroom -> bedroom
  bedroom -> bathroom
  bedroom -> livingroom
  kitchen -> livingroom
  livingroom -> bedroom
  livingroom -> kitchen

=====
Enter goal (or 'quit'):
```

Supported commands:	
at robot <room>	→ e.g. at robot kitchen
at <obj> <room>	→ e.g. at apple bedroom
holding <obj>	→ e.g. holding remote
emptyhand	→ drop current object